

BÀI 8. KHẢO SÁT HỆ THỐNG SOC VỚI BỘ DMA

1. Mục tiêu

Thông qua bài thực hành này, sinh viên sẽ biết:

- Cấu trúc bộ DMA của Intel (các thanh ghi, chức năng...).
- Cách xây dựng hệ thống phần cứng bằng Qsys để giao tiếp với bộ DMA.
- Cách xây dựng chương trình C trên công cụ Nios II – EDS để giao tiếp với bộ DMA, và sử dụng ngắt của bộ DMA.

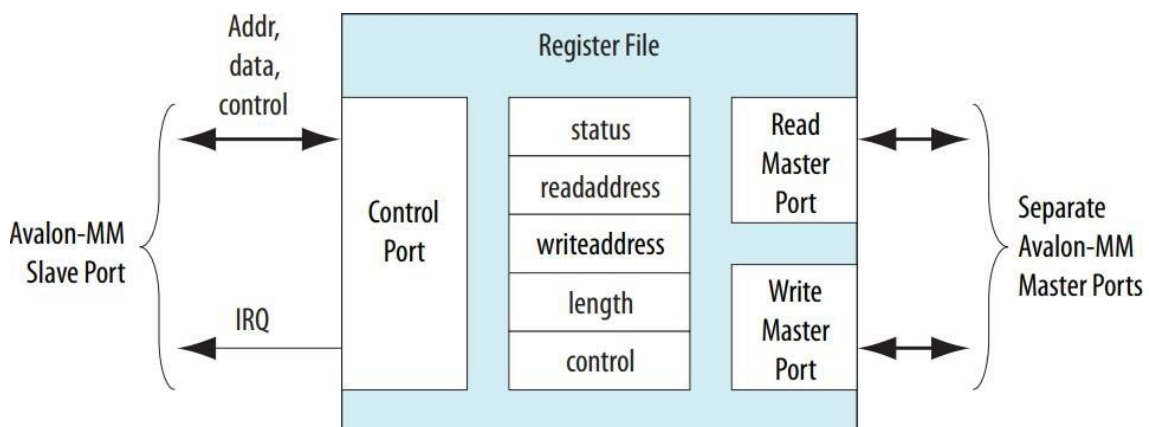
2. Phần lý thuyết

2.1. Lý thuyết về bộ DMA

2.1.1. Giới thiệu

DMA (Direct Memory Access) là một cơ chế truyền dữ liệu trực tiếp giữa hai hay nhiều thành phần trong hệ thống mà không thông qua CPU. Nhờ vậy, các quá trình truyền nhận dữ liệu có thể được thực hiện song song với tốc độ cao. Bộ DMA Controller của Altera thực hiện cơ chế DMA trong hệ thống SoC được xây dựng trên Platform Designer.

DMA Controller tiến hành việc truyền dữ liệu từ địa chỉ nguồn (source address) sang địa chỉ đích (destination address). Địa chỉ nguồn hoặc đích có thể là địa chỉ của ngoại vi hoặc bộ nhớ. Ngoài ra, DMA Controller còn hỗ trợ báo hiệu ngắt khi quá trình DMA hoàn tất.



Hình 1. Kiến trúc mô đun DMA.

DMA Controller gồm hai Avalon-MM master ports (master read port và master writeport) và một Avalon-MM slave port dùng để điều khiển DMA như hình bên trên.

DMA Controller hỗ trợ truyền theo kiểu pipeline hoặc burst với độ rộng dữ liệu từ 1 – 16 bytes.

2.1.2. Thanh ghi của bộ DMA

Module DMA bao gồm 5 thanh ghi cơ bản như bên dưới.

Offset	Register Name	Read / Write	31 13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	status (1)	RW	(2)									LEN	WEOP	REOP	BUSY	DONE
1	read address	RW	Read master start address													
2	write address	RW	Write master start address													
3	length	RW	DMA transaction length (in bytes)													
4	—	—	Reserved (3)													
5	—	—	Reserved (3)													
6	control	RW	(2)	SOF TWA RER ESE T	QUA DWO RD	DOUBLE WORD	WC ON	RC ON	LEE N	WEE N	REE N	I_E N	GO	WORD	HW	BYTE
7	—	—	Reserved (3)													
Notes : 1. Writing zero to the status register clears the LEN, WEOP, REOP, and DONE bits. 2. These bits are reserved. Read values are undefined. Write zero. 3. This register is reserved. Read values are undefined. The result of a write is undefined.																

a. Thanh ghi control

Bit Number	Bit Name	Read/Write/Clear	Description
0	BYTE	RW	Specifies byte transfers.
1	HW	RW	Specifies halfword (16-bit) transfers.
2	WORD	RW	Specifies word (32-bit) transfers.
3	GO	RW	Enables DMA transaction. When the GO bit is set to 0 during idle stage (before execution starts), the DMA is prevented from executing transfers. When the GO bit is set to 1 during idle stage and the length register is non-zero, transfers occur. If go bit is de-asserted low before write transaction complete, done bit will never go high. It is advisable that GO bit is modified during idle stage (no execution happened) only.
4	I_EN	RW	Enables interrupt requests (IRQ). When the I_EN bit is 1, the DMA controller generates an IRQ when the status register's DONE bit is set to 1. IRQs are disabled when the I_EN bit is 0.
5	REEN	RW	Ends transaction on read-side end-of-packet. When the REEN bit is set to 1, a slave port with flow control on the read side may end the DMA transaction by asserting its end-of-packet signal. REEN bit should be set to 0.
6	WEEN	RW	Ends transaction on write-side end-of-packet. WEEN bit should be set to 0.
7	LEEN	RW	Ends transaction when the length register reaches zero. When this bit is 0, length reaching 0 does not cause a transaction to end. In this case, the DMA transaction must be terminated by an end-of-packet signal from either the read or write master port. LEEN bit should be set to 1.
8	RCON	RW	Reads from a constant address. When RCON is 0, the read address increments after every data transfer. This is the mechanism for the DMA controller to read a range of memory addresses. When RCON is 1, the read address does not increment. This is the mechanism for the DMA controller to read from a peripheral at a constant memory address. For details, see the Addressing and Address Incrementing section.
9	WCON	RW	Writes to a constant address. Similar to the RCON bit, when WCON is 0 the write address increments after every data transfer; when WCON is 1 the write address does not increment. For details, see Addressing and Address Incrementing .
10	DOUBLEWORD	RW	Specifies doubleword transfers.
11	QUADWORD	RW	Specifies quadword transfers.
12	SOFTWARERESET	RW	Software can reset the DMA engine by writing this bit to 1 twice. Upon the second write of 1 to the SOFTWARERESET bit, the DMA control is reset identically to a system reset. The logic which sequences the software reset process then resets itself automatically.

b. Thanh ghi status

Bit Number	Bit Name	Read/Write/Clear	Description
0	DONE	R/C	A DMA transaction is complete. The DONE bit is set to 1 when an end of packet condition is detected or the specified transaction length is completed. Write zero to the status register to clear the DONE bit.
1	BUSY	R	The BUSY bit is 1 when a DMA transaction is in progress.
2	REOP	R	The REOP bit is 1 when a transaction is completed due to an end-of-packet event on the read side.
3	WEOP	R	The WEOP bit is 1 when a transaction is completed due to an end of packet event on the write side.
4	LEN	R	The LEN bit is set to 1 when the length register decrements to zero.

c. Thanh ghi readaddress

Chỉ ra địa chỉ đọc đầu tiên của quá trình truyền nhận dữ liệu. Giá trị “readaddress” phải sắp hàng theo độ rộng dữ liệu được cấu hình ở thanh ghi “control” (bit 0, 1, 2, 10, 11).

d. Thanh ghi writeaddress

Chỉ ra địa chỉ ghi đầu tiên của quá trình truyền nhận dữ liệu. Giá trị “writeaddress” phải sắp hàng theo độ rộng dữ liệu được cấu hình ở thanh ghi “control” (bit 0, 1, 2, 10, 11).

e. Thanh ghi length

Chỉ ra số lượng byte cần truyền nhận. Giá trị “length” phải là bội số của dữ liệu với độ rộng được cấu hình ở thanh ghi “control” (bit 0, 1, 2, 10, 11).

2.1.3. Hoạt động của DMA

Khi bắt đầu hoạt động, bộ DMA sẽ đọc dữ liệu tại địa chỉ trong thanh ghi “readaddress”, và ghi giá trị đọc được vào địa chỉ trong thanh ghi “writeaddress”. Độ rộng dữ liệu của quá trình đọc ghi được cấu hình trong thanh ghi “control” ở các bit 0, 1, 2, 10, và 11. Quá trình DMA kết thúc khi số lượng byte đã được đọc ghi bằng với số lượng byte được cấu hình trong thanh ghi “length”, lúc này:

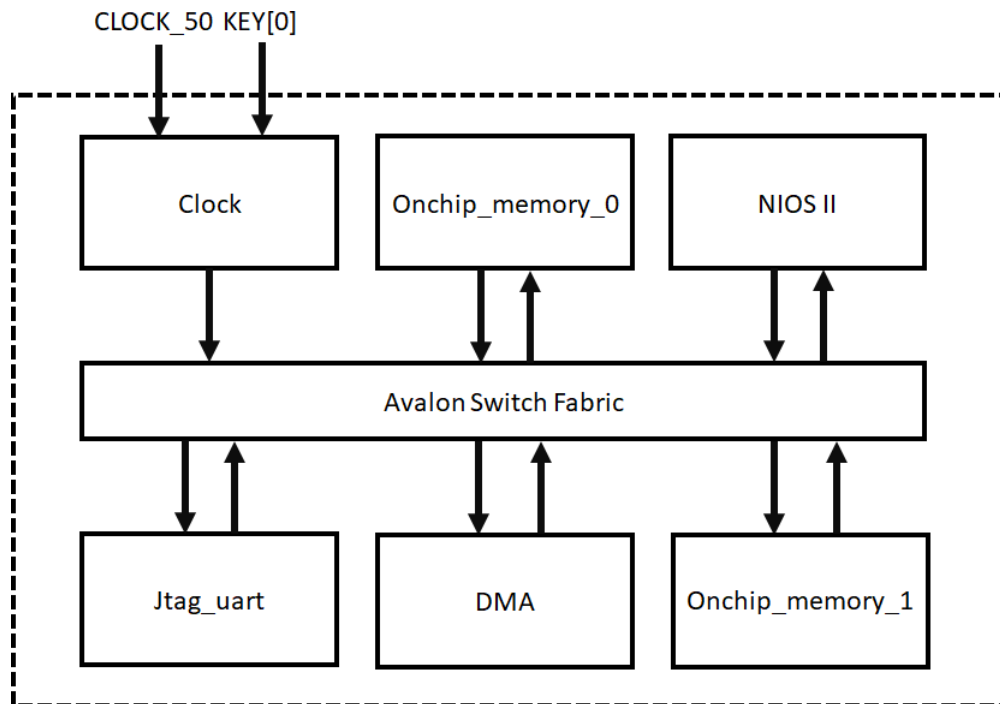
- Bit “LEN” và “DONE” của thanh ghi “status” sẽ bằng 1.
- Ngắt sẽ được sinh ra nếu bit “I_EN” của thanh ghi “control” bằng 1.

Trong hàm xử lý ngắt DMA, xóa ngắt bằng cách xóa bit “LEN” và “DONE” của thanh ghi “status”.

3. Phần thực hành

3.1. Tổng quan hệ thống

Hệ thống được thiết kế như hình 2 bên dưới.



Hình 2. Tổng quan hệ thống.

3.2. Tạo project Quartus Prime

- Tạo project Quartus tên là “**Bai8**”. Lưu ý đường dẫn thư mục project không được có khoảng trắng.
- Chọn Family là **Cyclone II E**, device là **EP2C35F672C6** nếu dùng board **DE2**.

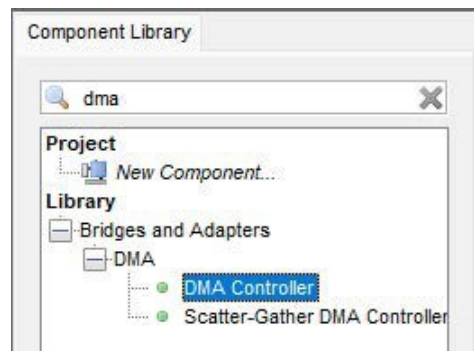
3.3. Xây dựng phần cứng trên Platform Designer

- Xây dựng hệ thống phần cứng như hình 3 bên dưới.

System Contents Address Map Clock Settings Project Settings Instance Parameters System Inspector HDL Example Generation									
Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
☒		clk_0 clk_in clk_in_reset clk_reset clk_reset nios2_qsys_0 clk reset_n data_master instruction_master jtag_debug_module_re jtag_debug_module custom_instruction_m onchip_memory2_0 clk1 s1 reset1 jtag_uart_0 clk reset avalon_jtag_slave	Clock Source Clock Input Reset Input Clock Output Reset Output Nios II Processor Clock Input Reset Input Avalon Memory Mapped Master Avalon Memory Mapped Master Reset Output Avalon Memory Mapped Slave Custom Instruction Master On-Chip Memory (RAM or ROM) Clock Input Avalon Memory Mapped Slave Reset Input JTAG UART Clock Input Reset Input Avalon Memory Mapped Slave	clk reset Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk] [clk] [clk] [clk] clk_0 [clk] [clk1] [clk1] [clk] [clk] [clk] [clk] [clk] [clk] [clk] [clk] [clk] [clk]	0x0001_1800 0x0000_8000 0x0001_2028	0x0001_1fff 0x0000_ffff 0x0001_202f	IRQ 0	

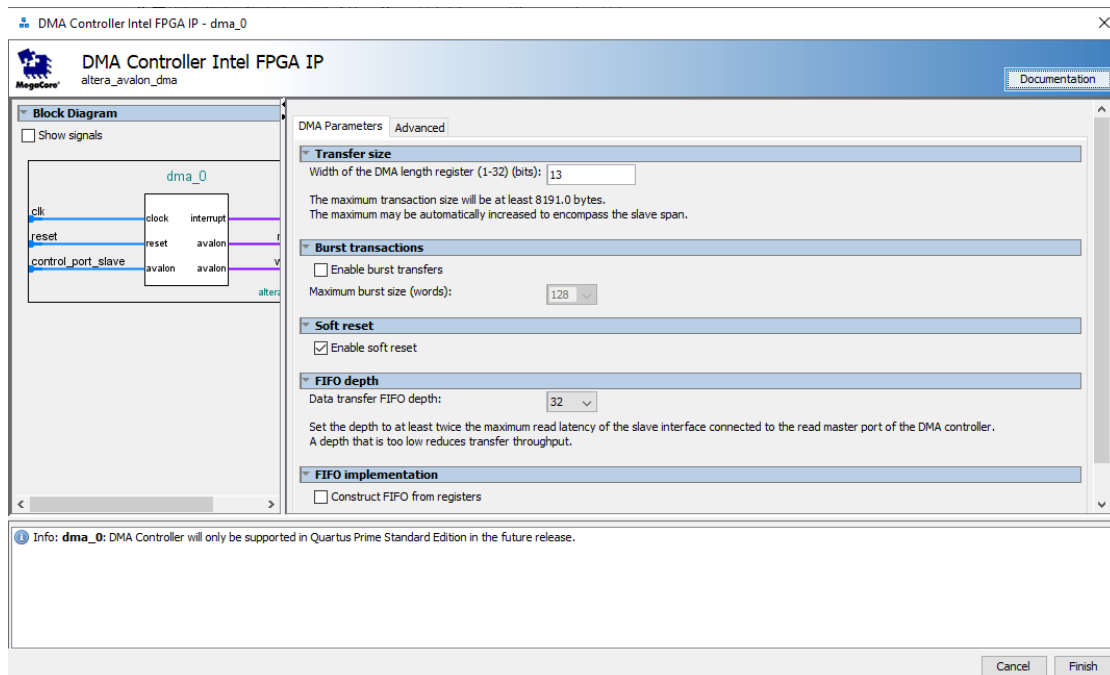
Hình 3. Phân cứng chuẩn bị ban đầu.

- Thêm mô đun DMA Controller bằng cách gõ DMA Controller vào ô tìm kiếm, và nhấn đúp chuột như hình 4.



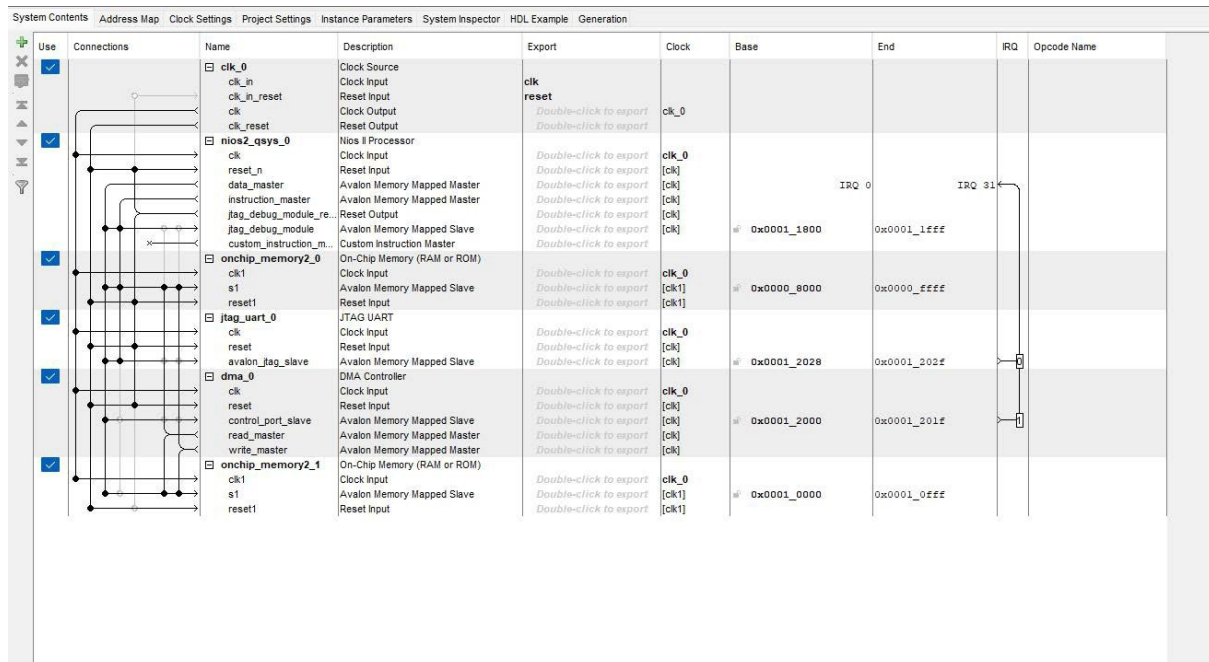
Hình 4. Thêm mô đun DMA ở tab IP Catalog.

- Giữ nguyên cấu hình như hình 5.



Hình 5. Cấu hình mặc định của mô đun DMA.

- Kết nối mô đun DMA vào hệ thống như hình 5.



Hình 6. Hệ thống hoàn chỉnh.

- Gán lại địa chỉ: **System** → **Assign Base Addresses**.
- Lưu hệ thống dưới tên là **system** và **generate** hệ thống.

3.4. Tích hợp hệ thống

- Thêm file **system.qip**.
- Tạo file top-level là **Bai8.v** được mô tả như đoạn code bên dưới để tổng hợp hệ thống.

```
module Bai8(
    input          CLOCK_50,
    input [0:0]    KEY
);
    system Nios_system(
        .clk_clk    (CLOCK_50),
```



```
.reset_reset_n (KEY[0])
);
endmodule
```

- Gán chân cho thiết bị: **Assignments** → **Import Assignments**.
- Tiến hành biên dịch project và cuối cùng là nạp xuống board.

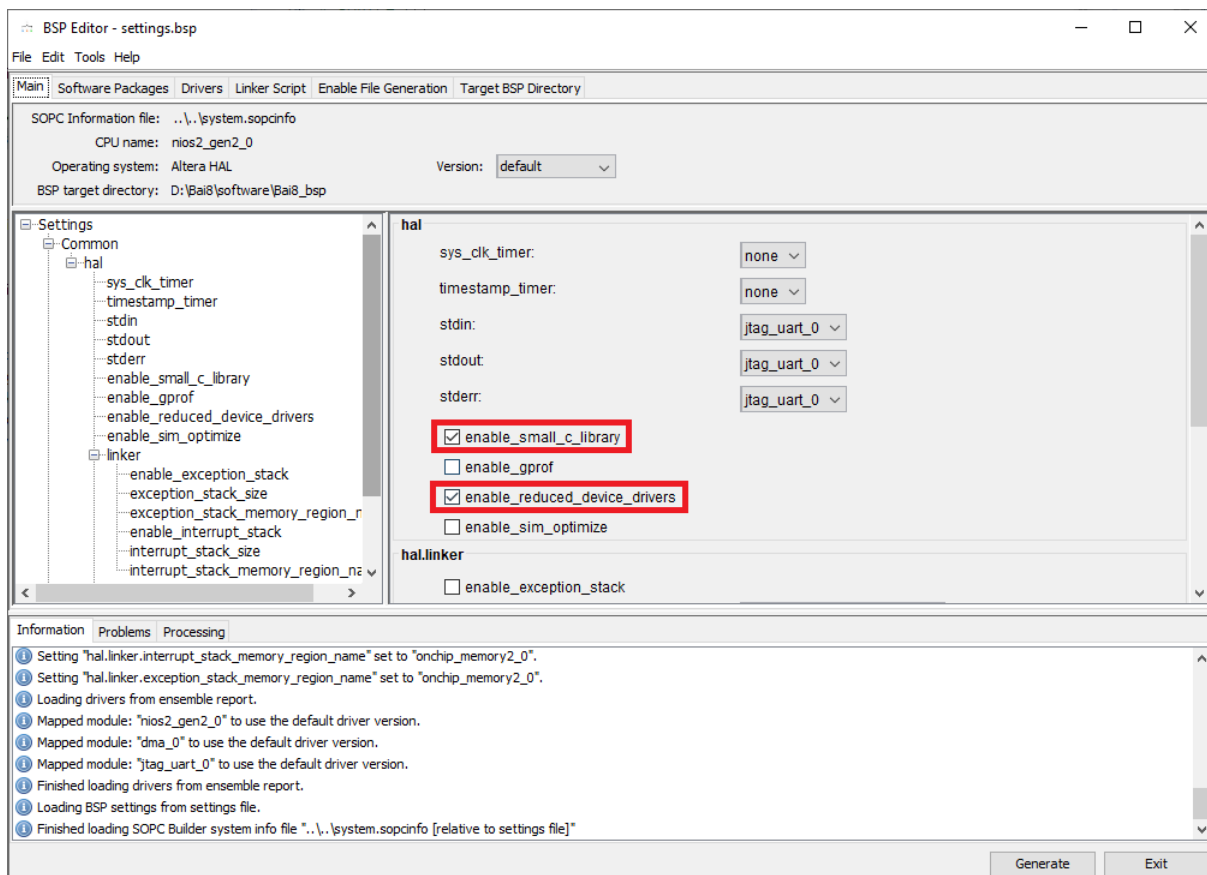
3.5. Xây dựng phần mềm

- Tạo và đặt tên project là “**Bai8**”.
- Thêm file “**source.c**” vào project “**Bai7**”. File “**source.c**” có nội dung như đoạn code bên dưới.

```
#include <stdio.h>
#include "system.h"
#include "altera_avalon_dma_regs.h"
#include "sys/alt_irq.h"
//pdata0 points to a global array stored in onchip_memory2_0
char pdata0[32] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31 };
//pdata1 points to onchip_memory2_1
char *pdata1 = (char*) (ONCHIP_MEMORY2_1_BASE);
//Interrupt handler of DMA
void DMA_ISR_Handler(void* isr_context) {
    int i;
    // Read and print data in onchip_memory2_1
    printf("DMA transfer complete. Data in destination memory:\n");
    for (i = 0; i < 32; i++) {
        printf("byte %d = %d\n", i, pdata1[i]);
    }
    //Clear DMA Interrupt bit
    IOWR_ALTERA_AVALON_DMA_STATUS(DMA_0_BASE, 0);
}
//Initialize function of DMA
void DMA_Init(void) {
    // De-init DMA
    IOWR_ALTERA_AVALON_DMA_CONTROL(DMA_0_BASE, 0);
    // Source address is pdata0
    IOWR_ALTERA_AVALON_DMA_RADDRESS(DMA_0_BASE, (int)pdata0);
    // Destination address is pdata1
    IOWR_ALTERA_AVALON_DMA_WADDRESS(DMA_0_BASE, (int)pdata1);
    // Length is 32 bytes
    IOWR_ALTERA_AVALON_DMA_LENGTH(DMA_0_BASE, 32);
    // Configure and Start DMA with byte transfer, end transaction when
    //length reach zero, interrupt enable and start DMA
    IOWR_ALTERA_AVALON_DMA_CONTROL(DMA_0_BASE,
        ALTERA_AVALON_DMA_CONTROL_BYTE_MSK |
        ALTERA_AVALON_DMA_CONTROL_LEEN_MSK | ALTERA_AVALON_DMA_CONTROL_I_EN_MSK |
        ALTERA_AVALON_DMA_CONTROL_GO_MSK);
}
int main() {
    DMA_Init();
    alt_ic_isr_register(DMA_0_IRQ_INTERRUPT_CONTROLLER_ID, DMA_0_IRQ,
        DMA_ISR_Handler, NULL, NULL);
}
```

```
while (1) {  
}  
  
return 0;  
}
```

- Trong thư mục “drivers/inc” của project “Bai8_bsp” có file “altera_avalon_dma_regs.h”. Đây là file định nghĩa các thanh ghi DMA và các bit của các thanh ghi này. Chúng ta sẽ sử dụng file này để cấu hình DMA.
- Cấu hình **BSP** như hình 7 bên dưới.



Hình 7. Cấu hình BSP.

- Build project và download phần mềm xuống board. Xem kết quả trên console.



Hình 8. Kết quả trên console.

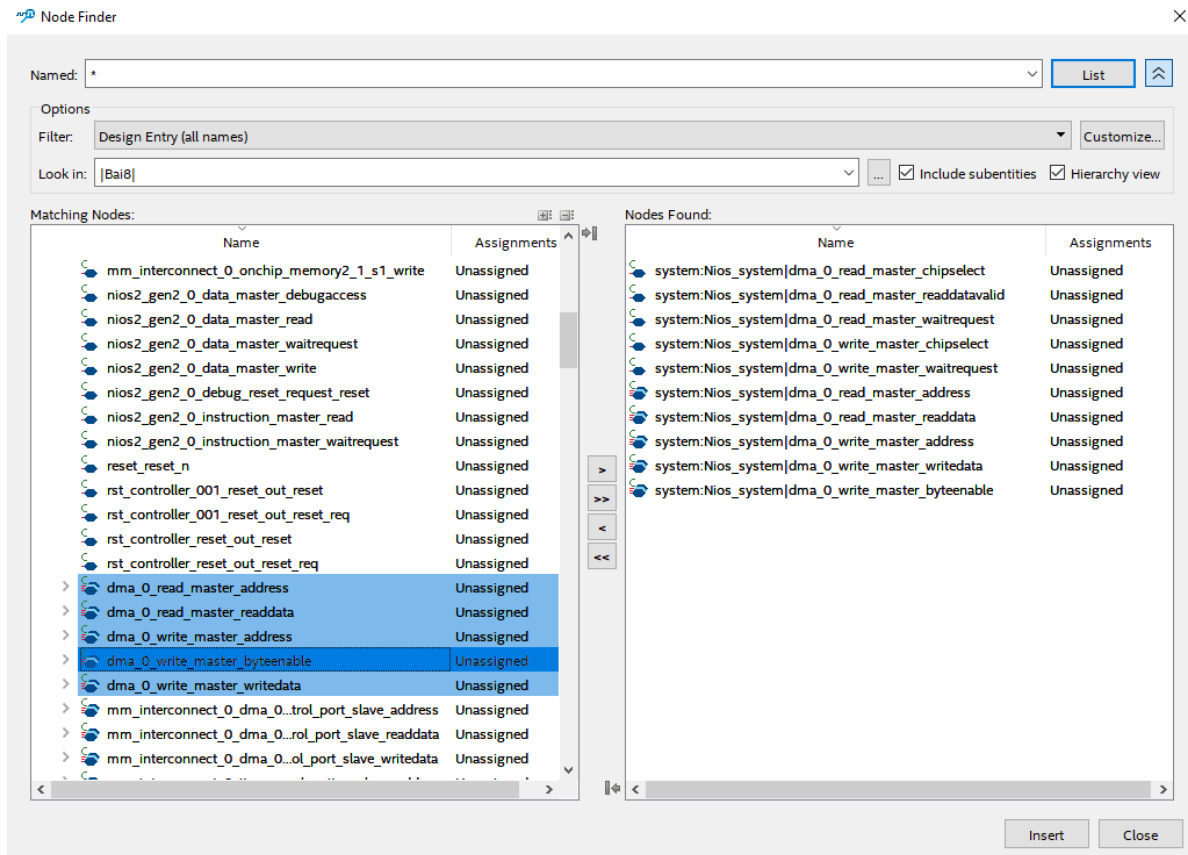
BÀI TẬP CHUẨN BỊ Ở NHÀ

Bài 1. Giải thích các thanh ghi và các bit của bộ DMA.

BÀO CÁO THỰC HÀNH

Bài 1. Tiến hành thực hiện khảo sát bộ DMA trên Signal Tab ở các cấu hình truyền “halfword transfer” và “word transfers” đọc 32 byte dữ liệu từ bộ nhớ “onchip_memory2_0” và ghi vào bộ nhớ “onchip_memory2_1”.

Gợi ý, bắt dữ liệu ở các tín hiệu như hình sau:



TÀI LIỆU THAM KHẢO

- [1] DE2_115_User_Manual.
- [2] Embedded Peripherals IP User Guide.
- [3] Nios II Processor Reference.
- [4] Avalon Interface Specification.